CHRIST (Deemed to be University)
Department of Computer Science
Master of Computer Applications

Course: MCA411-2 - PYTHON Programming
LAB Exercise III –
Demonstrate use of object- oriented programming concepts.

Date of assignment: 29-09-2026
Date of submission: 29-09-2026

Name : Aditya Kumar

Reg. No : 2647203
Class : 2 MCA – B

Domain : Clean Energy Systems & Grid Decarbonization

Lab 3

Lab Exercise II:

Demonstrate use of object- oriented programming concepts.

A. Develop a Python application for your domain by demonstrating the major Object-Oriented Programming (OOP) concepts. The program should be designed using a meaningful, real-world, domain-specific scenario and must demonstrate the following:

- **a. Class and Object**
- **b. Constructors**
- **c. Encapsulation**
- **d. Inheritance**
- **e. Polymorphism**
- **f. Abstraction**
- **g. Class and Static Methods**
- **h. Composition/Aggregation**
- **i. Special/Magic Methods**

Code:

```
'''
# Domain: Clean Energy Systems & Grid Decarbonization
'''


from abc import ABC, abstractmethod


'''
*** 1. OOP CLASSES: Base Abstract Class & Concrete Assets ***
'''

# 1. Abstraction (Abstract Class & Method) + Encapsulation + Constructor
class EnergyAsset(ABC):
    # Constructor to initialize attributes
    def __init__(self, asset_id, capacity_kw):
        self.asset_id = str(asset_id).strip().replace(" ", "").upper()
        self.__capacity_kw = float(capacity_kw)  # Private attribute
(Encapsulation)

    # Getter for capacity
    @property
    def capacity(self):
        return self.__capacity_kw

    # Setter with validation
    @capacity.setter
    def capacity(self, val):
        if val > 0:
            self.__capacity_kw = float(val)
        else:
```

```
            print("Error: Capacity must be positive.")

    # Abstract method
    @abstractmethod
    def calculate_output(self, sunlight_factor=1.0):
        pass


# 2. Inheritance (Level 1) + Polymorphism + Magic Methods
class SolarPanel(EnergyAsset):
    # Constructor initializing child and parent attributes
    def __init__(self, asset_id, capacity_kw, efficiency_pct=18.0):
        super().__init__(asset_id, capacity_kw)
        self.efficiency_pct = float(efficiency_pct)
        self.readings = []

    # Method Overriding & Overloading-like behavior via default arguments
    def calculate_output(self, sunlight_factor=1.0):
        return round(self.capacity * float(sunlight_factor) * (self.efficiency_pct
/ 100.0), 2)

    # Log new power output reading
    def log_reading(self, kw):
        if kw >= 0:
            self.readings.append(kw)

    # Magic Method: User-friendly string representation
    def __str__(self):
        return f"[{self.asset_id}] Solar Panel | Cap: {self.capacity} kW | Eff:
{self.efficiency_pct}%"

    # Magic Method: Number of readings logged
    def __len__(self):
        return len(self.readings)

    # Magic Method: Less-than comparison based on capacity
    def __lt__(self, other):
        return self.capacity < other.capacity

    # Magic Method: Equality check comparing asset IDs
    def __eq__(self, other):
        if isinstance(other, EnergyAsset):
            return self.asset_id == other.asset_id
        return self.asset_id == str(other).strip().replace(" ", "").upper()


# 3. Inheritance (Level 2 / Multilevel)
class SmartSolarPanel(SolarPanel):
    # Constructor for multilevel inherited class
    def __init__(self, asset_id, capacity_kw, efficiency_pct, tracking=True):
        super().__init__(asset_id, capacity_kw, efficiency_pct)
        self.tracking = tracking

    # Overriding calculation with tracking bonus
    def calculate_output(self, sunlight_factor=1.0):
        boost = 1.15 if self.tracking else 1.0
        return round(super().calculate_output(sunlight_factor) * boost, 2)
```

```
# 4. Aggregation / Composition + Class & Static Methods
class MicrogridSubstation:
    grid_standard = "IEEE-1547"  # Class attribute

    # Constructor initializing substation name and asset container
    def __init__(self, site_name):
        self.site_name = site_name
        self.assets = {}  # Direct Dictionary Storage: ID -> Object (Aggregation)

    # Add new asset to substation
    def add_asset(self, asset):
        key = asset.asset_id
        if key in self.assets:
            print(f"Error: {key} already exists.")
            return False
        self.assets[key] = asset
        print(f"{key} added successfully.")
        return True

    # Search asset by ID
    def get_asset(self, raw_id):
        # Cleans whitespace and hidden characters completely
        clean_key = str(raw_id).strip().replace(" ", "").upper()
        # Direct lookup first
        if clean_key in self.assets:
            return self.assets[clean_key]
        # Fallback partial matching
        for k, obj in self.assets.items():
            if k == clean_key or clean_key in k:
                return obj
        return None

    # Class Method
    @classmethod
    def get_standard(cls):
        return f"Grid Standard: {cls.grid_standard}"

    # Static Method
    @staticmethod
    def is_safe_temp(temp_c):
        return temp_c <= 45.0


'''
*** 2. Substation Initialization & Helper Functions ***
'''

substation = MicrogridSubstation("Bengaluru Substation East")

# Initial Objects
substation.add_asset(SolarPanel("SOLAR_01", 150.0, 18.0))
substation.add_asset(SmartSolarPanel("SOLAR_02", 90.0, 20.0, tracking=True))
substation.add_asset(SolarPanel("SOLAR_03", 200.0, 19.5))


# View all assets in the substation
def view_all_assets():
    print(f"\n--- Substation Assets ({substation.site_name}) ---")
    if not substation.assets:
```

```
        print("No assets found.")
        return

    for key, a in substation.assets.items():
        print(f"{a} | Logged Readings: {len(a)}")


# Register a new asset with validation
def add_new_asset():
    print("\n--- Register New Solar Asset ---")
    raw_id = input("Enter Asset ID (e.g., SOLAR_04): ").strip()
    clean_id = raw_id.replace(" ", "").upper()

    if clean_id == "":
        print("Asset ID cannot be empty.")
        return

    if substation.get_asset(clean_id) is not None:
        print(f"Error: Asset ID {clean_id} is already registered.")
        return

    cap_str = input("Enter Capacity in kW (10 to 500): ").strip()
    if not (cap_str.replace(".", "", 1).isdigit() and 10 <= float(cap_str) <=
500):
        print("Invalid capacity value.")
        return

    eff_str = input("Enter Efficiency % (10 to 30): ").strip()
    if not (eff_str.replace(".", "", 1).isdigit() and 10 <= float(eff_str) <= 30):
        print("Invalid efficiency percentage.")
        return

    is_smart = input("Enable Smart Sun-Tracking? (y/n): ").strip().lower()
    if is_smart == "y":
        panel = SmartSolarPanel(clean_id, float(cap_str), float(eff_str),
tracking=True)
    else:
        panel = SolarPanel(clean_id, float(cap_str), float(eff_str))

    substation.add_asset(panel)


# Search by ID and update capacity
def search_and_update():
    print("\n--- Search & Update Asset ---")
    user_input = input("Enter Asset ID: ").strip()
    asset = substation.get_asset(user_input)

    if asset is None:
        print(f"Asset '{user_input}' not found.")
        print("Available Asset IDs:", list(substation.assets.keys()))
        return

    print(f"\nFound Record: {asset}")
    ch = input("Do you want to update capacity? (y/n): ").strip().lower()
    if ch == "y":
        cap_str = input("Enter New Capacity in kW (10 to 500): ").strip()
        if cap_str.replace(".", "", 1).isdigit() and 10 <= float(cap_str) <= 500:
            asset.capacity = float(cap_str)  # Property setter
```

```
            print(f"Updated successfully: {asset}")
        else:
            print("Invalid capacity value.")


# Simulate power generation across objects
def simulate_generation():
    print("\n--- Simulate Generation Output ---")
    factor_str = input("Enter Sunlight Factor (0.0 to 1.0, e.g., 0.8): ").strip()
    if not (factor_str.replace(".", "", 1).isdigit() and 0.0 <= float(factor_str)
<= 1.0):
        print("Invalid sunlight factor.")
        return

    factor = float(factor_str)
    for a in substation.assets.values():
        output = a.calculate_output(factor)
        a.log_reading(output)
        print(f"{a.asset_id} Generated: {output} kW")


# Demonstrate static, class, and magic comparison methods
def test_oop_features():
    print("\n--- OOP Concepts Demonstration ---")
    print(substation.get_standard())
    print("Static Method (32°C safe?):", MicrogridSubstation.is_safe_temp(32.0))
    print("Static Method (48°C safe?):", MicrogridSubstation.is_safe_temp(48.0))

    asset_list = list(substation.assets.values())
    if len(asset_list) >= 2:
        a1 = asset_list[0]
        a2 = asset_list[1]
        print(f"\nComparing Capacities (__lt__):")
        print(f"Is {a1.asset_id} ({a1.capacity} kW) < {a2.asset_id} ({a2.capacity}
kW)?: {a1 < a2}")
        print(f"Equality Check (__eq__):")
        print(f"Is {a1.asset_id} == {a2.asset_id}?: {a1 == a2}")



# Main Console Menu
while True:
    print("\n--- SOLAR-BESS OBJECT CONSOLE ---")
    print("1. View All Assets")
    print("2. Add New Asset")
    print("3. Search & Update Asset Capacity")
    print("4. Simulate Generation (Polymorphism)")
    print("5. Test Magic, Class & Static Methods")
    print("6. Exit")

    choice = input("Enter option (1-6): ").strip()
    if choice == "1":
        view_all_assets()
    elif choice == "2":
        add_new_asset()
    elif choice == "3":
        search_and_update()
    elif choice == "4":
        simulate_generation()
```

```
    elif choice == "5":
        test_oop_features()
    elif choice == "6":
        print("Exiting...")
        break
    else:
        print("Invalid option.")
```

Output:

```
SOLAR_01 added successfully.
SOLAR_02 added successfully.
SOLAR_03 added successfully.

--- SOLAR-BESS OBJECT CONSOLE ---
1. View All Assets
2. Add New Asset
3. Search & Update Asset Capacity
4. Simulate Generation (Polymorphism)
5. Test Magic, Class & Static Methods
6. Exit
Enter option (1-6):
```

```
Enter option (1-6): 1

--- Substation Assets (Bengaluru Substation East) ---
[SOLAR_01] Solar Panel | Cap: 150.0 kW | Eff: 18.0% | Logged Readings: 0
[SOLAR_02] Solar Panel | Cap: 90.0 kW | Eff: 20.0% | Logged Readings: 0
[SOLAR_03] Solar Panel | Cap: 200.0 kW | Eff: 19.5% | Logged Readings: 0
```

```
Enter option (1-6): 2

--- Register New Solar Asset ---
Enter Asset ID (e.g., SOLAR_04): SOLAR_04
Enter Capacity in kW (10 to 500): 450
Enter Efficiency % (10 to 30): 25
Enable Smart Sun-Tracking? (y/n): y
SOLAR_04 added successfully.
```

```
Enter option (1-6): 3

--- Search & Update Asset ---
Enter Asset ID: SOLAR_04

Found Record: [SOLAR_04] Solar Panel | Cap: 450.0 kW | Eff: 25.0%
Do you want to update capacity? (y/n): y
Enter New Capacity in kW (10 to 500): 300
Updated successfully: [SOLAR_04] Solar Panel | Cap: 300.0 kW | Eff: 25.0%
```

```
Enter option (1-6): 4

--- Simulate Generation Output ---
Enter Sunlight Factor (0.0 to 1.0, e.g., 0.8): 0.7
SOLAR_01 Generated: 18.9 kW
SOLAR_02 Generated: 14.49 kW
SOLAR_03 Generated: 27.3 kW
SOLAR_04 Generated: 60.37 kW
```

```
Enter option (1-6): 5

--- OOP Concepts Demonstration ---
Grid Standard: IEEE-1547
Static Method (32°C safe?): True
Static Method (48°C safe?): False

Comparing Capacities (__lt__):
Is SOLAR_01 (150.0 kW) < SOLAR_02 (90.0 kW)?: False
Equality Check (__eq__):
Is SOLAR_01 == SOLAR_02?: False
```